React JS Developer Assessment

React JS Developer Assessment

Section 1. Background (FYI Only)

This section provides context. It is not part of the evaluation.
You are building a spa and wellness booking management system used by operations teams across multiple outlets.

Each outlet has:

- Up to 200 therapists
- Up to 2000 bookings per day
- Multiple services per booking
- Real-time schedule management

The UI consists of:

- A calendar schedule view
- Calendar must support drag and drop booking rescheduling
- Booking creation and editing forms
- Booking status updates
    - Confirmed [Blue color card]
    - Check-in (In Progress) [Pink color Card]
    - Cancelled [Grey Color Card]

Design references are provided in the Figma files at bottom of the page.

Backend APIs will be provided for the following operations:

- List bookings
- Create booking
- Update booking
- Delete booking

Your task is to build the frontend system only.

Section 2. Technical Requirements (Assessment Starts Here)

1. Build a Single Page Application -
React 19.0.0
Create React App
JavaScript (.js)

The application must contain:

- Calendar booking board
- Booking creation form
- Booking editing form
- Booking cancellation
- Booking details panel
- Search functionality

Total UI complexity includes around 10 forms/dropdowns.

Examples:

- Create booking
- Update booking
- Cancel booking
- Check-in/checkout
- Add service
- Add therapist
- Select room
- Select request type
- Notes
- Client search/creation
- Filters

2. Booking Calendar UI

Build a therapist schedule calendar similar to the provided design.

Calendar characteristics:

- Time grid (15-minute interval)
- Therapists displayed horizontally
- Time displayed vertically
- Bookings rendered as blocks

Example scale requirements:

- 2000 bookings per day
- 200 therapists
- 30 outlets (No need to worry for now)

Your UI must be designed so that it does not lag under this load.

Important considerations:

- Virtual rendering
- Efficient DOM updates
- Avoid unnecessary re-renders
- Memory management

3. API Integration

You will be provided APIs in the POSTMAN COLLECTION. Please look at bottom of the page for API collection.

GET /bookings

POST /bookings/create

PUT /bookings/{id}

CANCEL /bookings/item/cancel

DELETE /bookings/destroy/{id}

And more…

The UI must:

- Fetch bookings
- Display bookings on the calendar
- Create new bookings
- Update existing bookings
- Delete or cancel bookings

4. Color/icons Rules

When displaying therapist labels:

- Female → Pink
- Hex: #EC4899
- Male → Blue
- Hex: #3B82F6

Example:

Lily (Female) → Pink label

James (Male) → Blue label

You can put requested therapist icon at letter **T**

You can put requested room icon at letter **R**

5. Booking Panel

When clicking a booking block, open a right side panel.
Panel must allow:

- View booking

- Edit booking
- Cancel booking
- Select service
- Select therapist
- Update duration

6. Local Data Handling
Your solution must support:

- Local caching of bookings
- Instant UI updates in real time when creating and updating the booking
- No UI lag

Test scenario:

Load 2000 bookings per day in the calendar.
The UI must remain responsive.

7. Error Handling

Your application must demonstrate proper handling of:

Frontend errors

Examples:

- Invalid form input
- Failed state updates
- Component rendering failures

Backend errors

Examples:

- API timeout
- Validation failure
- Server error

Errors must be:

- Logged
- Displayed in UI
- Gracefully recoverable

8. Logging Strategy

Implement a logging mechanism that captures:

- API errors
- UI exceptions
- User actions

Example events:

- Booking created
- Booking edited
- Booking cancelled
- API failure

Logs can be printed to the console or structured locally.

9. Performance Expectations

The application should demonstrate knowledge of:

- Memoization
- Code splitting
- Lazy loading
- Optimized rendering

You should avoid:

- Full re-renders of the calendar
- Heavy state propagation
- Unnecessary component updates

10. Architecture Expectations

We expect the candidate to demonstrate:

- Component modularity
- Clean folder structure
- Reusable components
- Clear state management

State management can use:

- Context API
- Redux
- Zustand

Submission Requirements
Provide:

1. Hosted working link
   (Vercel / Netlify / similar)

2. Short documentation

Include:

- Architecture overview
- State management explanation
- Performance strategy
- Assumptions made

Do NOT submit code repositories initially. We don't need your code. We will review the link, and you will be invited to an interview where we can collaboratively review the code from your screenshare. We don't need your code.

If shortlisted, you will be invited to:

- Walk through your architecture
- Explain performance decisions
- Demonstrate your code locally from your computer

Evaluation Criteria

We will evaluate the following:

- Architecture quality
- Performance under load
- State management strategy
- Code structure
- Error handling
- UI responsiveness
- Problem-solving approach

Time Expectation

Expected completion time:

07 Days

The required collection is available here.

Figma
https://www.figma.com/design/MdKG8Sk1Zuw3CX4XFWhaNn/SPA-booking-system?node-id=0-1&p=f&t=L4Kqny9bhqgSC2zN-0

API Collection

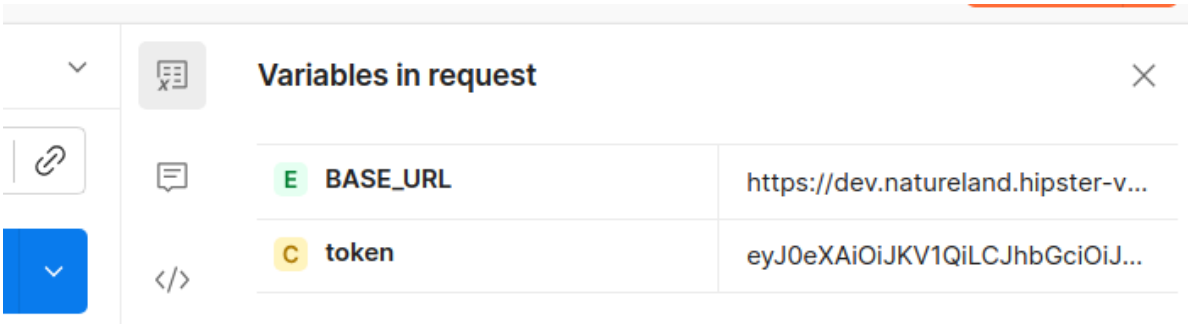
https://api.postman.com/collections/19902956-08daf3c0-ddc4-4bec-aa6a-d50cbaa9c5b3?access_key=PMAT-01KMAWM5DT932JRBNGXKAF0SQ4

BaseURL: https://dev.natureland.hipster-virtual.com {{BASE_URL}}

Full Endpoint example: {{BASE_URL}}/api/v1/login

Parameter: You can find it in api itself.

Auth: Every API is authenticated. We already have added {{**token**}} which is used in each api. So you can get it from the login api and put it into the variable.



Important Note

We have Google Captcha in the login form. You can bypass it by using the information below in the login api.

Email: react@hipster-inc.com

Password: React@123

Key_pass: 07ba959153fe7eec778361bf42079439